

ADVANCED DATABASE ORGANIZATION
PROGRAMMING ASSIGNMENT IV: INDEX
MANAGER

Contribution Table

CWID	Name	Contribution Description	Percent Contribution
A20594489	Kori Kamoljonov	Designed and implemented the B+ Tree insert, search, delete, and scan operations. Integrated buffer pool.	33.3%
A20568393	Chaitanya Datta	Developed B+ Tree scan sorting and expression evaluation. Assisted with testing and edge case handling.	33.3%
A20582646	Vamshi Krishna	Worked on buffer management, test case development, and tree metadata handling and storage integration.	33.3%

1. Task

The goal of this assignment is to implement an *Index Manager* using a *B+ Tree* data structure. The system should support operations such as insertion, deletion, search, and scan on integer-based keys. This index manager interfaces with the previously built buffer and storage managers to persist index data. Each key in the index is associated with a record ID (RID), and all operations ensure correct and efficient handling of these key-RID mappings.

- The B+ Tree supports fixed-size integer keys only.
- The page file stores the B+ Tree metadata and nodes.
- Keys are uniquely inserted. Duplicate insertions return errors.
- The tree structure is static and unbalanced for simplicity.
- Pages are buffered and flushed using the buffer manager.

To ensure consistent and reliable testing, your implementation should follow the following conventions:

- **To split a leaf node during insertion**, follow these steps:

If the maximum number of keys (n) is even, the extra key should go to the left node.

For example, if $n = 2$ and we insert the key 4 into the node $[1, 5]$, the resulting leaf nodes will be $[1, 4]$ and $[5]$.

If n is odd, the keys can be distributed equally between the two nodes.

In both cases, the value inserted into the parent node should correspond to the right node's smallest key.

- **When splitting a non-leaf node**

with an odd n , the keys cannot be divided evenly. One of the new nodes will include an additional key.

In this case, the middle value inserted into the parent should originate from the correct node.

For example, if $n = 3$ and the non-leaf node $[1, 3, 4, 5]$ needs to be split, the output should be $[1, 3]$ and $[5]$, with 4 inserted into the parent.

- **leaf underflow**

your implementation should first try redistributing keys from a sibling node. Only if redistribution is not possible should the node be merged with a sibling.

Both redistribution and merging should give preference to the left sibling.

For instance, if keys can be borrowed from both left and right siblings, the implementation should prioritize borrowing from the left.

2. Index Manager Interface.

2.1. Index Manager Initialization and Shutdown

These functions are in charge of initializing the index manager and releasing all acquired resources at shutdown. They ensure that the system is properly configured before performing any index operations, and that memory or file handles are properly cleaned up when no longer required.

2.2. B+-Tree operations

This collection of functions provides the fundamental functionality for **managing B+-tree indexes**:

Functions allow you to **create new B+-tree indexes** or **delete existing ones**. When you delete an index, you should also delete the page file that corresponds to it.

2.3 Index Manager Interface

To access a B+-tree, clients must first open it with **openBtree()**.

When the tree is no longer in use, **call closeBtree()**. During this process, the index manager must ensure that all newly created or modified pages are flushed to disk using the correct buffer manager functions.

2.4. Key Management Functions.

These functions allow clients to search for, insert, and delete keys within a B+-tree index, as well as browse entries in sorted order.

- **findkey()** searches for a specific key in the B+-tree and returns the corresponding **Record ID (RID)**.
If the key cannot be found, the function should return the **RC_IM_KEY_NOT_FOUND** error code (defined in dberror.h).
- **insertkey()** adds a new key-record pointer pair to the index.
If the key is already present, the function should return the error code **RC_IM_KEY_ALREADY_EXISTS**.

B+ Tree Scanning

Clients can iterate over all entries in the B+ tree in sorted order using the following scan-related functions:

- **openTreeScan()** — Starts a scan operation.
- **nextEntry()** — Returns the next entry in sorted order.
- **closeTreeScan()** — Ends the scan.

When the scan reaches the tree's end, the **nextEntry()** function should return the error code **RC_IM_NO_MORE_ENTRIES**.

2.5. Debug Functions.

The `printTree` function generates a string representation of a B+ tree. It is essential for testing and debugging and must generate output in a specific format to be compatible with test cases. Each node in the B+-tree should be represented on its own line in the output string. Nodes must be printed in the depth-first pre-order traversal.

key: Each key should be serialized with the **`serializeValue` function** from the previous assignment.

pointer: Refers to **the positions of child nodes** in the depth-first pre-order traversal (not physical disk addresses).

3. FILES

3.1 TABLES.H

The `tables.h` header is reused from the Record Manager to define several essential data types:

- **Value:** Represents typed data used to store keys and evaluate conditions. While the structure supports **DT-STRING**, **DT FLOAT**, and **DT BOOL**, only **DT INT** is utilized in this assignment.
- **RID:** Record Identifier used to associate keys with actual records (page number + slot number).
- **Schema and Record:** While not directly used for full schema operations, they play a minimal role in expression evaluation.

These structures ensure compatibility with the broader database architecture and facilitate scan-based comparisons using `expr.h`. They also enable serialization and future extensibility into the Record Manager pipeline.

```

1  #ifndef TABLES_H
2  #define TABLES_H
3
4  #include "dt.h"
5
6  // Data Types, Records, and Schemas
7  typedef enum DataType {
8      DT_INT = 0,
9      DT_STRING = 1,
10     DT_FLOAT = 2,
11     DT_BOOL = 3
12 } DataType;
13
14 typedef struct Value {
15     DataType dt;
16     union v {
17         int intV;
18         char *stringV;
19         float floatV;
20         bool boolV;
21     } v;
22 } Value;
23
24 typedef struct RID {
25     int page;
26     int slot;
27 } RID;
28
29 typedef struct Record
30 {
31     RID id;
32     char *data;
33 } Record;
34
35 // information of a table schema: its attributes, datatypes,
36 typedef struct Schema
37 {
38     int numAttr;
39     char **attrNames;
40     DataType *dataTypes;
41     int *typeLength;
42     int *keyAttrs;
43     int keySize;
44 } Schema;
45
46 // TableData: Management Structure for a Record Manager to handle one relation
47 typedef struct RM_TableData
48 {
49     char *name;
50     Schema *schema;
51     void *mgmtData;
52 } RM_TableData;
53
54 #define MAKE_STRING_VALUE(result, value) \
55     do { \
56         (result) = (Value *) malloc(sizeof(Value)); \
57         (result)->dt = DT_STRING; \
58         (result)->v.stringV = (char *) malloc(strlen(value) + 1); \
59         strcpy((result)->v.stringV, value); \
60     } while(0)
61
62 #define MAKE_VALUE(result, datatype, value) \
63     do { \
64         (result) = (Value *) malloc(sizeof(Value)); \
65         (result)->dt = datatype; \
66         switch(datatype) \
67         { \
68             case DT_INT: \
69                 (result)->v.intV = value; \
70                 break; \
71             case DT_FLOAT: \
72                 (result)->v.floatV = value; \
73                 break; \
74             case DT_BOOL: \
75                 (result)->v.boolV = value; \
76                 break; \
77             } \
78     } while(0)
79
80
81 // debug and read methods
82 extern Value *stringToValue (char *value);
83 extern char *serializeTableInfo(RM_TableData *rel);
84 extern char *serializeTableContent(RM_TableData *rel);
85 extern char *serializeSchema(Schema *schema);
86 extern char *serializeRecord(Record *record, Schema *schema);
87 extern char *serializeAttr(Record *record, Schema *schema, int attrNum);
88 extern char *serializeValue(Value *val);
89 extern RC attrOffset (Schema *schema, int attrNum, int *result);
90
91
92 #endif
93
94

```

3.2 EXPR.H

Expressions are evaluated during scan operations. The `expr.h` header defines expression trees used for filtering keys in the B+ Tree during scans.

Key Components:

- **Expression Types:** Constants, Attribute References, and Operators.
- **Operators:** Logical (AND, OR, NOT) and Comparison (EQUAL, LESS THAN).
- **evalExpr():** Evaluates whether a record satisfies a condition.

Although the full attribute reference system (**EXPR ATTRREF**) isn't used in this assignment, the **EXPR CONST** type and comparison logic (**OP COMP SMALLER**, etc.) are used to implement conditional scans. This modular approach allows the scan to evolve into more complex filtering logic in future versions

```

1  #ifndef EXPR_H
2  #define EXPR_H
3
4  #include "dberror.h"
5  #include "tables.h"
6
7  // datatype for arguments of expressions used in conditions
8  typedef enum ExprType {
9      EXPR_OP,
10     EXPR_CONST,
11     EXPR_ATTRREF
12 } ExprType;
13
14 typedef struct Expr {
15     ExprType type;
16     union expr {
17         Value *cons;
18         int attrRef;
19         struct Operator *op;
20     } expr;
21 } Expr;
22
23 // comparison operators
24 typedef enum OpType {
25     OP_BOOL_AND,
26     OP_BOOL_OR,
27     OP_BOOL_NOT,
28     OP_COMP_EQUAL,
29     OP_COMP_SMALLER
30 } OpType;
31
32 typedef struct Operator {
33     OpType type;
34     Expr **args;
35 } Operator;
36
37 // expression evaluation methods
38 extern RC valueEquals (Value *left, Value *right, Value *result);
39 extern RC valueSmaller (Value *left, Value *right, Value *result);
40 extern RC boolNot (Value *input, Value *result);
41 extern RC boolAnd (Value *left, Value *right, Value *result);
42 extern RC boolOr (Value *left, Value *right, Value *result);
43 extern RC evalExpr (Record *record, Schema *schema, Expr *expr, Value **result);
44 extern RC freeExpr (Expr *expr);
45 extern void freeVal(Value *val);
46
47 #define CPVAL(_result,_input) \
48 do { \
49     (_result)->dt = _input->dt; \
50     switch(_input->dt) \
51     { \
52         case DT_INT: \
53             (_result)->v.intV = _input->v.intV; \
54             break; \
55         case DT_STRING: \
56             (_result)->v.stringV = (char *) malloc(strlen(_input->v.stringV) + 1); \
57             strcpy((_result)->v.stringV, _input->v.stringV); \
58             break; \
59         case DT_FLOAT: \
60             (_result)->v.floatV = _input->v.floatV; \
61             break; \
62         case DT_BOOL: \
63             (_result)->v.boolV = _input->v.boolV; \
64             break; \
65     } \
66 } while(0)
67
68 #define MAKE_BINOP_EXPR(_result,_left,_right,_optype) \
69 do { \
70     Operator *_op = (Operator *) malloc(sizeof(Operator)); \
71     _result = (Expr *) malloc(sizeof(Expr)); \
72     _result->type = EXPR_OP; \
73     _result->expr.op = _op; \
74     _op->type = _optype; \
75     _op->args = (Expr **) malloc(2 * sizeof(Expr*)); \
76     _op->args[0] = _left; \
77     _op->args[1] = _right; \
78 } while (0)
79
80 #define MAKE_UNOP_EXPR(_result,_input,_optype) \
81 do { \
82     Operator *_op = (Operator *) malloc(sizeof(Operator)); \
83     _result = (Expr *) malloc(sizeof(Expr)); \
84     _result->type = EXPR_OP; \
85     _result->expr.op = _op; \
86     _op->type = _optype; \
87     _op->args = (Expr **) malloc(sizeof(Expr*)); \
88     _op->args[0] = _input; \
89 } while (0)
90
91 #define MAKE_ATTRREF(_result,_attr) \
92 do { \
93     _result = (Expr *) malloc(sizeof(Expr)); \
94     _result->type = EXPR_ATTRREF; \
95     _result->expr.attrRef = _attr; \
96 } while(0)
97
98 #define MAKE_CONS(_result,_value) \
99 do { \
100     _result = (Expr *) malloc(sizeof(Expr)); \
101     _result->type = EXPR_CONST; \
102     _result->expr.cons = _value; \
103 } while(0)
104
105
106
107
108 #endif // EXPR
109

```

3.3 BTREE_MGR.H

This header defines the API of the B+ Tree Index Manager. Core functionalities are organized into the following categories:

3.3.1 Initialization and metadata management

- **initIndexManager() / shutdownIndexManager()** – Handles global index environment setup and teardown.
- **createBtree()** – Creates a new index file, initializes metadata, and sets up the root node.
- **openBtree() / closeBtree()** – Opens or closes an existing B+ Tree file and links it to a BTreeHandle.
- **deleteBtree()** – Permanently deletes an index file.

3.3.2 Key Operations

- **insertKey()** – Adds a new key and associates it with a record ID. If the current node is full, a new node is created and linked. Node splitting is not implemented, so the tree grows in a flat, linked-list fashion.
- **findKey()** – Searches for a key in all available nodes and returns its corresponding RID.
- **deleteKey()** – Marks a key-RID pair as deleted by setting default values. Deleted slots are skipped during scans.

3.3.3 Scan Operations

- **openTreeScan()** – Collects all valid entries and prepares them for sorted traversal.
- **nextEntry()** – Iterates through sorted key-RID pairs.
- **closeTreeScan()** – Frees up memory used by the scan handle.

Scan logic uses a global array and performs bubble sort to enable ordered traversal of keys. This approach is efficient for small datasets but may need optimization for scalability.

3.3.4 Utility Functions

- **getNumNodes() / getNumEntries()** – Returns B+ Tree statistics.
- **printTree()** – Generates a human-readable string representation of all tree nodes, useful for debugging and visualization.


```

1  #ifndef BTREE_MGR_H
2  #define BTREE_MGR_H
3
4  #include "dberror.h"
5  #include "tables.h"
6
7  // structure for accessing btrees
8  typedef struct BTreeHandle {
9      DataType keyType;
10     char *idxId;
11     void *mgmtData;
12 } BTreeHandle;
13
14 typedef struct BT_ScanHandle {
15     BTreeHandle *tree;
16     void *mgmtData;
17 } BT_ScanHandle;
18
19 // init and shutdown index manager
20 extern RC initIndexManager (void *mgmtData);
21 extern RC shutdownIndexManager ();
22
23 // create, destroy, open, and close an btree index
24 extern RC createBtree (char *idxId, DataType keyType, int n);
25 extern RC openBtree (BTreeHandle **tree, char *idxId);
26 extern RC closeBtree (BTreeHandle *tree);
27 extern RC deleteBtree (char *idxId);
28
29 // access information about a b-tree
30 extern RC getNumNodes (BTreeHandle *tree, int *result);
31 extern RC getNumEntries (BTreeHandle *tree, int *result);
32 extern RC getKeyType (BTreeHandle *tree, DataType *result);
33
34 // index access
35 extern RC findKey (BTreeHandle *tree, Value *key, RID *result);
36 extern RC insertKey (BTreeHandle *tree, Value *key, RID rid);
37 extern RC deleteKey (BTreeHandle *tree, Value *key);
38 extern RC openTreeScan (BTreeHandle *tree, BT_ScanHandle **handle);
39 extern RC nextEntry (BT_ScanHandle *handle, RID *result);
40 extern RC closeTreeScan (BT_ScanHandle *handle);
41
42 // debug and test functions
43 extern char *printTree (BTreeHandle *tree);
44
45 #endif // BTREE_MGR_H
46

```

4. Optional Extensions.

The B+-tree index manager's functionality and performance can be expanded by implementing the optional enhancements listed below:

- **Integration with the Record Manager.**

Modify the record manager to enable B+-tree-based searching:

When creating a new relation, allow the user to specify the table's key attribute. The record manager should automatically generate and maintain a B+-tree to store keys from this attribute. When a new record is created, the corresponding key should be added to the B+-tree. Searches for records by key should use the B+-tree rather than a full table scan. If composite keys (keys with multiple attributes) are supported, the B+-tree interface can be modified accordingly.

- **Support for multiple data types.**

Extend the B+-tree to handle additional data types (as described in Assignment 3), allowing for greater compatibility with various types of keys.

- **Enable Pointer Swizzling.**

When a B+-tree node is loaded into memory, replace conceptual pointers (such as page numbers) with memory pointers. In B+-trees, only parent nodes point to child pages, so swizzling can be used at page load to replace the pointer with a memory address. The challenge is to reverse the swizzling that occurs when the buffer manager evicts a page.

- Support for duplicate keys.
- Allow for multiple entries to be associated with the same key.
- Modify the design to allow for the insertion and retrieval of multiple **RIDs** per key.
- Keep the current **findKey()** function, which should only return the first matching entry.
- Create a new method, **findAllEntries()**, which returns a list or array of all entries for the specified keys
- Pointers in leaf nodes should point to a page containing all RIDs for that key rather than a single RID.
- If the list exceeds one page, create a linked list of pages, with each page pointing to the next.

4.1 Optional Extensions (Not implemented)

The advanced features listed below were identified as potential enhancements but were not implemented in the current version. The following could be considered for future development to improve the system's robustness, realism, and performance:

Node Splitting and Dynamic Tree Balancing

Automatic rebalancing mechanisms are used to maintain optimal tree structure following insertions or deletions.

SQL-style NULL Support and Null Bitmap Encoding

Allowing fields to store NULL values and efficiently managing them via bitmap encoding techniques.

Enforcing uniqueness of primary keys

Implementing strict enforcement to ensure that primary key constraints are not violated during insertion.

Tombstone Records and Transaction ID Tracking

Implementing tombstone markers and tracking transaction IDs to improve concurrency and recovery.

External Sorting of Massive Datasets

Using external sorting algorithms to handle datasets that exceed main memory capacity.

Interactive CLI Interface or Conditional Updates with `updateScan()`

Conditional updates and interactive usage are supported via a command-line interface for improved usability.

These features suggest promising directions for future research, especially in situations where scalability, data integrity, and user interaction are critical.

5. Source Code Structure

To maintain consistency and ease of evaluation, your project's source code should adhere to a well-defined directory structure. You are expected to reuse your current Storage Manager and Buffer Manager implementations. Before beginning development, move these existing modules to a new folder called `assign4`.

Required Directory Structure

All source files must be in the `assign4` folder of your Git repository. This folder should contain: The supplied header (.h) and C (.c) files. A Makefile to compile your implementation. All of your B+-tree source files (*.c and *.h), which implement the core index manager functionality. A README file (README.txt or README.md) containing a brief summary of your solution, including:

```
assign4/  
|-- btree_mgr.c / btree_mgr.h # B+ Tree core logic and API  
|-- buffer_mgr.c / buffer_mgr.h # Buffer Pool and page replacement strategies (e.g., FIFO)  
|-- buffer_mgr_stat.c / buffer_mgr_stat.h # Buffer pool content printing utilities  
|-- dberror.c / dberror.h # Error logging and return codes  
|-- expr.c / expr.h # Expression tree creation and evaluation logic  
|-- rm_serializer.c # Value and schema string serializers for debug output  
|-- storage_mgr.c / storage_mgr.h # Page file creation, reading, writing  
|-- tables.h # Common data structures: Value, RID, Schema, Record  
|-- test_assign4_1.c # Tests for insert, delete, find, scan functionality  
|-- test_expr.c # Tests for value comparison and boolean expression logic  
|-- Makefile # Compilation and linking instructions  
|-- README.md # Overview, build instructions, and usage examples
```

This structure supports reusability, modular debugging, and streamlined testing across multiple assignments.

6. Test Cases.

A comprehensive test suite is provided to ensure that the expression evaluation framework and the B+-tree index functionality are both correct and robust.

6.1. Expression Tests (test_expr.c).

These tests validate the expression evaluation system, which is especially important for scanning operations in the **Record Manager and B+-tree**.

Comparison Operators

Ensures proper behavior of `==`, `<`, `>`, and other operators for both integer and string values.

Boolean operations validate logical expressions with AND, OR, and NOT.

Nested Expressions tests complex expressions that include constants, variables, and logical/comparison operators.

These tests also **address edge cases**, such as:

1. Invalid or incompatible data types.
2. Missing or NULL values.

6.2. B+ Tree Index Tests (test_assign4_1.c)

This test suite focuses on validating all of the key functionalities of the B+-tree index implementation.

1. Insert & Find:

Confirms that inserted keys correctly return their associated RIDs.

Validates the internal structure by counting nodes and entries.

2. Delete:

Confirms that deleted keys are no longer accessible using scan or search operations.

Ensures proper rebalancing or merging behavior when applicable.

3. Scan:

To ensure complete in-order traversal of the tree, use **openTreeScan()** and **nextEntry()**.

Inserts keys in random order to test the structure's ability to handle unsorted input.

All tests are repeated using randomized data over multiple iterations to ensure:

Functional correctness, **Memory safety** and assertions like **ASSERT_EQUALS_RID()** and **ASSERT_EQUALS_INT()** are used to precisely validate expected results.